

INTERFACE – GIAO DIỆN

1

1

Định nghĩa

- ☐ Một giao diện là được khai báo bởi từ khóa Interface.
- ☐ Một giao diện không thực thi các hành động.
- ☐ Một giao diện có nghĩa là không dữ liệu, không cài đặt
- ☐ Một giao diện chỉ trưng ra các khả năng.

2

2

Tính chất

- ❑ Một giao diện có thể kế thừa một hoặc nhiều giao diện khác.
- ❑ Một lớp hay cấu trúc có thể cài đặt một hay nhiều giao diện.
- ❑ Về mặt lập trình thì giao diện là tập hợp các hàm được khai báo sẵn mà không cài đặt.

3

3

Cài đặt

```
interface name
{
    ret-type method-name1(param-list);

    ret-type method-name2(param-list);
    // ...

    ret-type method-nameN(param-list);
}
```

4

4

Thuộc tính Interface

- Dạng chung của mô tả thuộc tính Interface

```
// interface property
```

```
type name {
    get;
    set;
}
```

5

5

Thuộc tính Interface

- Một interface có thể được mô tả bởi chỉ mục.
- Một chỉ mục(indexer) được khai báo bên trong interface theo dạng sau:

```
// interface indexer
element-type this[int index] {
    get;
    set;
}
```

6

6

Hoạt động interface

- Dạng chung của một class thực thi một Interface:

```
class class-name : interface-name
{
    // class-body
}
```

7

7

CÚ PHÁP

```
public interface IList : ICollection, IEnumerable {
    int Add (object value);    // methods
    bool Contains (object value);
    ...
    bool IsReadOnly { get; } // property
    ...
    object this [int index] { get; set; }    // indexer
}
```

- Interface là abstract class; không có phần cài đặt.
- Có thể chứa **methods**, **properties**, **indexers** and **events** (không chứa trường dữ liệu, hằng, hàm tạo, hàm hủy, toán tử, kiểu lồng).
- Các thành phần của interface là *public abstract (virtual)*.
- Các thành phần của interface không thể là *static*.
- Class và struct có thể cài đặt nhiều interface.
- Interface có thể mở rộng các interface khác.

8

8

interface được cài đặt bởi class và struct

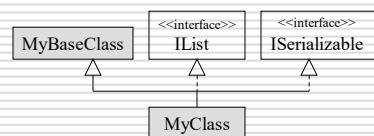
```
class MyClass : MyBaseClass, IList, ISerializable {
    public int Add (object value) {...}
    public bool Contains (object value) {...}
    ...
    public bool IsReadOnly { get {...} }
    ...
    public object this [int index] { get {...} set {...} }
}
```

- ❑ Một class chỉ kế thừa từ một baseclass, nhưng có thể cài đặt cho nhiều interface.
- ❑ Một struct không kế thừa, nhưng có thể cài đặt cho nhiều interface.
- ❑ Tất cả các thành phần của interface (method, property, indexer) đều phải được cài đặt hoặc được thừa kế.
- ❑ Khi cài đặt các phương thức của interface không cần sử dụng từ khoá *override*, có thể sử dụng từ khoá *virtual* hoặc *abstract*.
- ❑ Nếu phương thức Add của MyClass được chồng trong các subclass thì nó nên được khai báo với từ khoá *virtual*.

9

9

THỰC THI GIAO DIỆN



Gán: MyClass c = new MyClass();
 IList list = c;

Gọi phương thức: list.Add("Tom"); // gán động => myClass.Add

Kiểm tra kiểu: if (list is MyClass) ... // true
 if (list is ISerializable) ... // true

Ép kiểu: c = list as MyClass;
 c = (MyClass) list;
 ISerializable ser = (ISerializable) list;

10

10

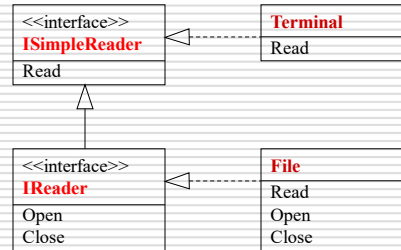
VÍ DỤ

```
interface ISimpleReader {
    int Read();
}

interface IReader : ISimpleReader {
    void Open(string name);
    void Close();
}

class Terminal : ISimpleReader {
    public int Read() { ... }
}

class File : IReader {
    public int Read() { ... }
    public void Open(string name) { ... }
    public void Close() { ... }
}
```



```
// null có thể được gán cho các biến của các kiểu interface
ISimpleReader sr = null;      sr = new Terminal();
sr = new File();

IReader r = new File();
sr = r;
```

11

11

XUNG ĐỘT TÊN

Xảy ra khi có hai interface cơ sở có phương thức cùng tên

```
interface I1 {
    void F();
}
interface I2 {
    void F();
}
class B : I1, I2 {
    //-----cài đặt phương thức F của class B
    public void F() { Console.WriteLine("B.F"); }
    //----- cài đặt phương thức F cho từng interface
    void I1.F() { Console.WriteLine("I1.F"); }
    void I2.F() { Console.WriteLine("I2.F"); } // phải là public
}

B b = new B();
b.F();           // B.F

I1 i1 = b;
i1.F();          // I1.F

I2 i2 = b;
i2.F();          // I2.F
```

12

12